Основы криптографии

Сессионное задание

1 Трехсторонний протокол распределения ключей

При трехстороннем распределении ключей участникам A и B помогает доверенный сервер S. Он создает общий ключ для их сеанса связи. Заранее A имеет общий секретный ключ K_{AS} с сервером, а B - ключ K_{BS}. Общего ключа между A и B пока нет. Рассмотрим симметричный протокол Нидхэма - Шрёдера.

Обозначим шифрование данных X ключом K через $E_K(X)$, новый сеансовый ключ - через K_{AB}. Числа N_A и N_B участники случайно выбирают для текущего обмена.

Шаг 1. A просит сервер выдать ключ для связи с B и передает свое случайное число:

$$A \rightarrow S : A, B, N_A.$$

Шаг 2. Сервер создает K_{AB} и отправляет A защищенный ответ. В него вложен отдельный билет для B:

$$S \rightarrow A : E_{K_{AS}}(N_A, B, K_{AB}, E_{K_{BS}}(K_{AB}, A)).$$

A расшифровывает ответ и проверяет N_A и имя B. Это связывает полученный ключ с текущим запросом и нужным собеседником. Билет A открыть не может, поскольку не знает K_{BS}.

Шаг 3. A пересылает билет участнику B:

$$A \rightarrow B : E_{K_{BS}}(K_{AB}, A).$$

B расшифровывает билет своим ключом и получает K_{AB} вместе с именем A.

Шаг 4. Чтобы проверить, знает ли A сеансовый ключ, B отправляет ему зашифрованное случайное число:

$$B \rightarrow A : E_{K_{AB}}(N_B).$$

Шаг 5. A расшифровывает число, уменьшает его на единицу и возвращает ответ:

$$A \rightarrow B : E_{K_{AB}}(N_B - 1).$$

B проверяет результат. Изменение числа не позволяет просто вернуть ему тот же вызов. После обмена A и B имеют общий симметричный ключ K_{AB} для сеанса связи. Сервер тоже знает его, поскольку сам его создал.

У исходного протокола есть ограничение: B не проверяет свежесть билета. Если старый сеансовый ключ раскрыт, злоумышленник может повторить билет и ответить на вызов. Поэтому нужны дополнительные проверки свежести, например срока действия билета, и защита целостности сообщений.

2 Формальные модели шифров

Формальная модель описывает сообщения, ключи и правила работы шифра. Разные модели позволяют проверить правильность расшифрования, оценить секретность и учесть вычислительные затраты.

Алгебраическая модель. Шифр задается набором $\Sigma = (\mathbf{M}, \mathbf{C}, \mathbf{K}, E, D)$. Здесь **M** - множество сообщений, **C** - криптограмм, **K** - ключей, а E и D - семейства функций шифрования и расшифрования. Для каждого ключа k требуется:

$$E_k : \mathbf{M} \to \mathbf{C}, \quad D_k : \mathbf{C} \to \mathbf{M}, \quad D_k(E_k(m)) = m.$$

Последнее равенство означает, что расшифрование возвращает исходное сообщение. Поэтому разные сообщения при одном ключе не должны давать одну криптограмму: E_k должно быть инъективным. Если конечные множества **M** и **C** одинакового размера, E_k также биективно. Например, блочный шифр при фиксированном ключе переставляет все блоки заданной длины.

Вероятностная модель. Сообщение M, ключ K и криптограмма C рассматриваются как случайные величины. Обычно ключ выбирают независимо от сообщения. Для детерминированного шифрования $C = E_K(M)$; при использовании случайных данных R получаем $C = E_K(M; R)$. Совершенная секретность означает:

$$\Pr[M = m \mid C = c] = \Pr[M = m] \quad \text{при } \Pr[C = c] > 0.$$

То есть криптограмма не меняет вероятности возможных сообщений для противника. Эта модель позволяет оценить, какую информацию раскрывает шифрование.

Алгоритмическая модель. Схема состоит из алгоритмов Gen, Enc и Dec. Gen создает ключ по параметру безопасности λ, Enc шифрует сообщение, а Dec восстанавливает его или сообщает об ошибке. Для симметричной схемы:

$$k \leftarrow \mathrm{Gen}(1^\lambda), \quad \mathrm{Dec}_k(\mathrm{Enc}_k(m; r)) = m.$$

Здесь r - случайные данные шифрования. В асимметричной схеме Gen выдает открытый ключ pk и закрытый ключ sk:

$$\mathrm{Dec}_{sk}(\mathrm{Enc}_{pk}(m; r)) = m.$$

Эта модель учитывает время и ресурсы, необходимые для работы алгоритмов и взлома. Правильность расшифрования сама по себе не означает стойкость: схема, просто передающая исходный текст, тоже расшифровывается без ошибок.

Автоматная модель. Шифр описывается как автомат с множеством состояний Q, текущим состоянием q_i, начальным состоянием q_0, функцией перехода f и функцией выхода g:

$$q_{i+1} = f(q_i, m_i, k), \quad c_i = g(q_i, m_i, k).$$

На каждом шаге автомат обрабатывает символ и меняет состояние. Получатель должен воспроизвести нужное состояние и обратить преобразование символа. Такая модель удобна для поточных шифров и режимов с обратной связью.

3 Принципы построения поточных систем шифрования

Поточный шифр обрабатывает текст по битам или небольшим группам битов. В распространенной схеме генератор создает ключевой поток - гамму z_i, которая складывается с текстом по модулю 2:

$$c_i = m_i \oplus z_i, \quad m_i = c_i \oplus z_i.$$

Повторный XOR с той же гаммой восстанавливает сообщение, поскольку $z_i \oplus z_i = 0$. Надежная система должна соблюдать следующие принципы.

Непредсказуемость. Зная часть гаммы, противник не должен уметь быстро вычислить ее продолжение или ключ. Одних хороших статистических свойств для этого недостаточно.

Достаточный размер ключа и состояния. Они должны затруднять перебор и восстановление генератора. Период гаммы должен превышать разрешенный объем данных одного запуска, чтобы последовательность не начала повторяться.

Нелинейность. Один простой линейный регистр сдвига не обеспечивает стойкости: его состояние можно восстановить по выходу. Нелинейные преобразования затрудняют такое восстановление, а также линейные и корреляционные атаки.

Правильная инициализация. Начальное состояние зависит от ключа и параметра инициализации. Его выбирают по правилам шифра: обычно требуется уникальный nonce, а некоторые режимы требуют непредсказуемого IV. Одну гамму нельзя использовать для двух сообщений, иначе:

$$c_i \oplus c'_i = m_i \oplus m'_i.$$

Синхронизация. Отправитель и получатель должны использовать одну позицию гаммы. В синхронном шифре она не зависит от текста и криптограммы, поэтому потеря данных может нарушить синхронизацию. В самосинхронизирующемся шифре гамма зависит от нескольких предыдущих символов шифртекста. Состояние со временем восстанавливается, но одиночная ошибка влияет на несколько следующих сегментов. Пример такого режима - CFB.

Скорость и целостность. Простые операции, такие как XOR, сдвиги и сложение, позволяют быстро обрабатывать данные с небольшим расходом памяти. При этом само гаммирование не обнаруживает подмену: изменение бита шифртекста может изменить бит сообщения. Поэтому системе нужна проверка тега аутентификации.

4 Режим без требования биективности базового отображения

Примером служит **режим счетчика CTR**. Для каждой позиции формируется известный обеим сторонам блок T_i, содержащий nonce и счетчик. Ключевая функция F_k превращает его в гамму:

$$c_i = m_i \oplus F_k(T_i), \quad m_i = c_i \oplus F_k(T_i).$$

Получатель вычисляет ту же прямую функцию F_k. Обратная функция ему не нужна. Подставив выражение для c_i, получим:

$$(m_i \oplus F_k(T_i)) \oplus F_k(T_i) = m_i.$$

Поэтому даже совпадение значений F_k на разных входах не делает расшифрование неоднозначным: вход и гамма для каждой позиции уже определены. Обратимым остается XOR текста с фиксированной гаммой.

В стандартном CTR используется блочный шифр, который является перестановкой. Но для правильности этой схемы биективность базовой функции не обязательна. Для стойкости нужны качественная псевдослучайная функция и неповторяющиеся входы T_i при одном

ключе. Например, нулевая гамма позволяет однозначно восстановить текст, но совсем его не скрывает.

5 Распространение ошибки в режиме CFB

Ответ: область возможного искажения охватывает два соседних 64-битовых блока. Считаем, что размер блока базового шифра и регистра равен b = 64 битам. Размер сегмента CFB равен s = 8 битам, то есть одному байту. После ошибки должно оставаться достаточно данных.

Шаг 1. Работа режима. Регистр R_i шифруется, из результата берут старшие восемь битов гаммы и складывают их с байтом шифртекста:

$$z_i = \mathrm{MSB}_8(E_k(R_i)), \quad m_i = c_i \oplus z_i.$$

Затем регистр сдвигается на один байт, а справа добавляется полученный c_i. Начальное состояние регистра - IV.

Шаг 2. Текущий байт. Пусть $c'_i = c_i \oplus e$, где e содержит один единичный бит. До этого шага регистр правильный, поэтому гамма не меняется:

$$m'_i = (c_i \oplus e) \oplus z_i = m_i \oplus e.$$

Значит, в текущем байте открытого текста изменится ровно тот же один бит.

Шаг 3. Следующие байты. Испорченный c'_i попадает в регистр и влияет на гамму следующих b/s = 64/8 = 8 сегментов. Каждый шаг сдвигает его на байт; после восьми шагов он выходит из регистра. Поэтому сегменты i + 1, …, i + 8 могут искажаться, а с i + 9 текст снова восстанавливается правильно. Всего затронуты:

$$1 + b/s = 1 + 64/8 = 9 \text{ байт}.$$

Шаг 4. Переход к 64-битовым блокам. Каждый такой блок содержит восемь байтов. Если ошибочный байт стоит на позиции r, где $1 \le r \le 8$, область влияния заканчивается на позиции r + 8. Поэтому число затронутых блоков:

$$N = \lceil (r + 8)/8 \rceil = 2, \quad \text{так как } 9 \le r + 8 \le 16.$$

Например, при ошибке в четвертом байте текущего блока в нем меняется один бит, а байты 5-8 могут искажаться. В следующем блоке могут искажаться байты 1-4; начиная с пятого байта расшифрование правильно. Получаются два соседних блока. При этом отдельные биты или байты могут случайно остаться правильными, а у конца сообщения область влияния будет короче.

6 Вероятность коллизии и длина хэша

Коллизия - совпадение хэшей разных сообщений. Примем модель идеального хэша: результаты для разных сообщений независимы, и все $N = 2^n$ значений равновероятны. Однонаправленность сама по себе этого не гарантирует, поэтому данное предположение необходимо для расчета.

Шаг 1. Вероятность совпадения. После выбора хэша первого сообщения у второго есть N равновероятных результатов. Только один совпадет с первым, значит:

$$p = 1/N = 2^{-n}.$$

Это частный случай модели дней рождений для двух сообщений: вероятность отсутствия совпадения равна 1 − 1/N, а вероятность коллизии - 1 − (1 − 1/N) = 1/N. Для множества сообщений пришлось бы учитывать число сравниваемых пар.

Шаг 2. Составим неравенство. По условию вероятность не должна превышать 10^{-9}:

$$2^{-n} \le 10^{-9} \quad \Rightarrow \quad 2^{n} \ge 10^{9}.$$

Берем логарифм по основанию 2. Он возрастает, поэтому знак неравенства сохраняется:

$$n \ge \log_2(10^9) = 9 \log_2 10 \approx 9 \cdot 3{,}321928095 \approx 29{,}897352855.$$

Число битов целое, поэтому округляем вверх: $n_{min} = 30$.

Шаг 3. Проверим границу. Для 29 и 30 битов получаем:

$$2^{-29} = 1/536\,870\,912 \approx 1{,}86265 \cdot 10^{-9} > 10^{-9},$$

$$2^{-30} = 1/1\,073\,741\,824 \approx 9{,}31323 \cdot 10^{-10} < 10^{-9}.$$

Следовательно, **минимальная длина хэша - 30 бит**; подходят все целые $n \ge 30$. Этот ответ относится к одной случайной паре различных сообщений в принятой модели равномерного хэша.